

Flowing containers in a liner shipping network

42115 Network Optimization

Teresa Christine Goldenits (s222604)

Ifigeneia Tziola (s222569)

Stefanie Patricia Freymann (s221641)

4 December 2022

Finding the cheapest route through a network

Question 1

The Bellman-Ford algorithm computes the shortest path from a source node to a destination node in a directed graph with weighted edges. In contrast to the Dijkstra algorithm, edge weights may be negative. However, there must be no negative cycles in a potential path from the start node. Such negative cycles can be detected by the algorithm, though. What the algorithm does not include are edge capacity constraints. Therefore, to map the path of a container of size w TEU with $w \in \{1, 2, 4\}$ over the given weighted, directed network with additional edge capacities $u_{ij} \geq 0$, the network must be modified before starting the algorithm. If $u_{ij} < w$, the edge must be deleted before the algorithm can be used. Furthermore, since the algorithm should output the cost of routing w TEU, all the edge costs must be multiplied by w .

Question 2

The implementation of the algorithm in Java can be found in the Appendix.

Question 3

After running the Java code, a total cost of 68 and a reverse path 4 - 2 - 1 - 5, i.e. the path 5 - 1 - 2 - 4 or E - A - B - D were identified by the algorithm. The input file and the console output are displayed in Figure 1.

```

BellmanFord.java
1 import java.io.File;
4
5 public class BellmanFord {
6
7     public static void main(String[] args) throws Exception {
8
9         //scanning of text file
10        //insert path for text file
11        File file = new File("C:\\Users\\spfre\\Desktop\\Bellmanfordtest.txt");
12        Scanner sc = new Scanner(file);
13
14        int n = sc.nextInt();
15        int m = sc.nextInt();
16        int s = sc.nextInt()-1;
17        int t = sc.nextInt()-1;
18        int w = sc.nextInt();
19        String Output0 = String.format("n = %d, m = %d, s = %d, t = %d, w = %d", n, m, s+1, t+1, w);
20        //System.out.println(Output0);
21
22        //create matrix for edges and respective edge info
23        int[][] edges = new int[m][4];
24
25        //check how many of these edges h
26        int reali = 0;
27
28        for (int i = 0; i < m; i++)
29        {
30            sc.nextLine();
31            int a = sc.nextInt();
32            int b = sc.nextInt();
33            int c = sc.nextInt();
34            int u = sc.nextInt();

```

```

Bellmanfordtest - ...
Datei Bearbeiten Format Ansicht Hilfe
5 9 5 4 4
1 2 8 14
2 3 7 13
3 4 8 9
4 5 1 8
5 1 7 16
1 4 1 2
4 1 1 2
2 4 2 10
4 2 2 3

```

```

Problems Javadoc Declaration Console
<terminated> BellmanFord [Java Application] C:\Users\spfre\p2\pool\
Cost: 68
Path (backwards): 4 - 2 - 1 - 5

```

Figure 1: Screenshot of console output and .txt-input of the code

Question 4

With the introduction of a maximum number of nodes h that may be visited, the Bellman-Ford algorithm can no longer be used. Likewise, adjusting the network before running the algorithm – like before when edge capacities had been introduced – cannot solve the problem. However, a shortest path with a restriction on the number of nodes visited can be computed using a resource constrained shortest path problem model. The upper bound for the resource can be interpreted as the upper bound for the total number of nodes visited. For this, the resource weight of each edge is set to 1. Finally, the resource constraint for the problem is set to $h - 1$. Since h comprises the start and end nodes and the number of edges in a path is the number of nodes in the path minus 1, the resource constraint must accordingly be adjusted to $h - 1$.

Finding bottlenecks in the network

Question 5

In order to find the largest amount of containers that can be sent from region S to T , we use a Maximum Flow Algorithm (e.g. Ford Fulkerson). Before we do that, we first have to modify the graph G , since there can be multiple sources and multiple sinks, in contrast to the standard form of the Max Flow Problem. In order to resolve this problem, we add a super-source s together with arcs that go from s to each node in S and a super-sink t together with arcs that go from each node in T to t . The amount that can flow through all these arcs is infinite because there is no upper bound stated in the problem description.

Question 6

In this case, there are 2 sources and 3 sinks. Therefore, we modify the graph as described in Question 5, adding a super-source s with the corresponding arcs (s, A) and (s, D) and a super-sink t with the corresponding arcs (B, t) , (C, t) and (E, t) , all these arcs having infinite capacities. The modified graph (with only the capacities displayed) can be found in Figure 2.

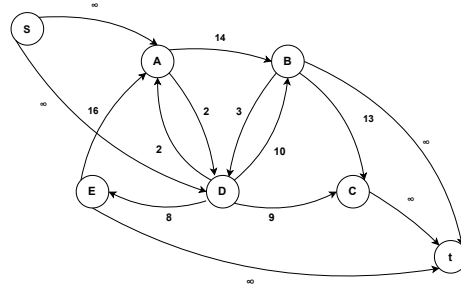


Figure 2: Modified graph with a super-source and a super-sink

We decided to apply the Ford-Fulkerson Algorithm in order to solve the Maximum Flow Problem for the modified graph. The augmenting paths can be found below for each step and are visualized in Figures 3, 4 and 5.

- Step 1: Path $sABCt$, capacity 13
- Step 2: Path $sADet$, capacity 2
- Step 3: Path $sABDEt$, capacity 1
- Step 4: Path $sDCt$, capacity 9
- Step 5: Path $sDBt$, capacity 10
- Step 6: Path $sDEt$, capacity 5

The final amount that flows from s to t is 40 TEU. Because of the way we have modified the graph in Question 5, the maximum flow from S to T will be the same as the maximum flow from s to t and therefore equal to 40.

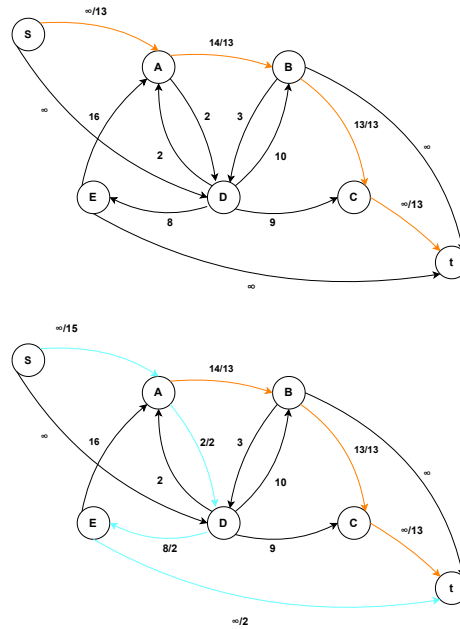


Figure 3: Ford Fulkerson – Steps 1 and 2

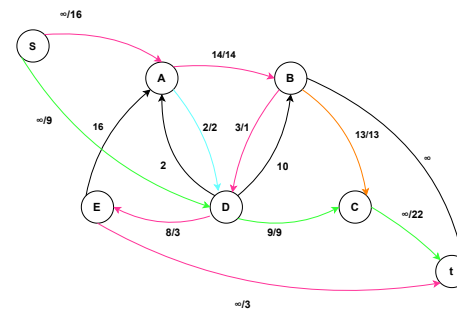
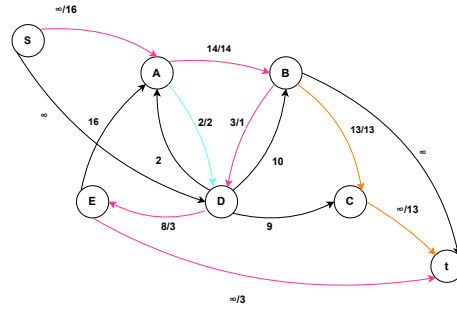


Figure 4: Ford Fulkerson – Steps 3 and 4

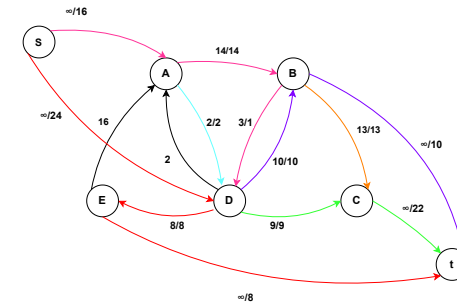
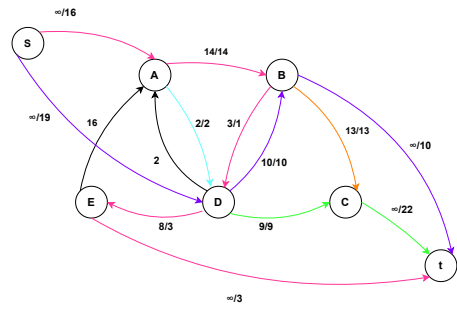


Figure 5: Ford Fulkerson – Steps 5 and 6

Question 7

To find the maximum number of edge disjoint paths between two ports s and t , we first set the capacity u_{ij} of each edge $(i, j) \in E$ to 1 and then apply a Max Flow algorithm (e.g. Ford-Fulkerson) to this modified graph. By setting the edge capacities to 1, each edge can be used only once in the maximum flow found by the algorithm, and therefore the paths contained in it must be edge disjoint. Since the flow is maximum, the number of paths found by the algorithm must also be maximum in this case.

Question 8

We calculate the maximum number of edge disjoint paths from port $s = B$ to port $t = A$ by applying the Ford-Fulkerson algorithm to the MeditLine graph that has been modified as described in Question 7 and which can be found in Figure 6.

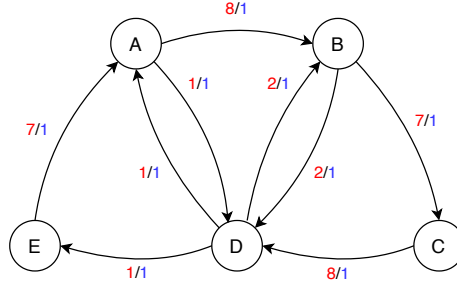


Figure 6: Modified graph G for finding edge disjoint paths

In the first step of the algorithm, we find an augmenting path from A to B : BDA . By inspecting the residual graph, we can see that there is another augmenting path: $BCDEA$. Now there are no vertices j left with $(B, j) \in E_x$, which means that we cannot find another augmenting path. In total, we can state that the maximum number of edge disjoint paths from A to B is 2. The steps of the algorithm are visualized in Figure 7.

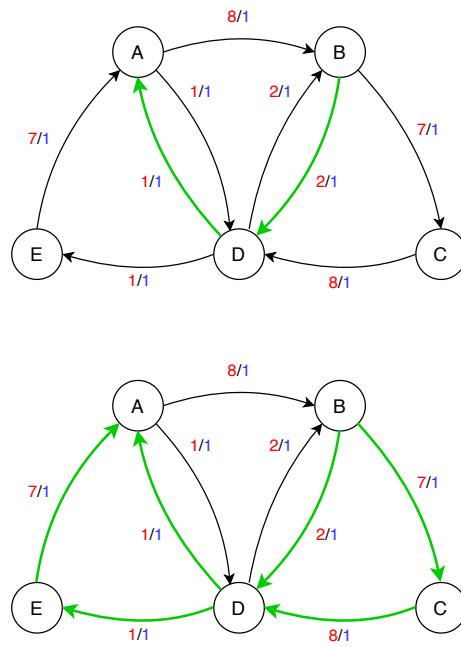


Figure 7: Application of Ford-Fulkerson to modified graph G

Repositioning of empty containers

Question 9

We formulate the problem as a Min Cost Flow problem by adding the demands d_i to the nodes. The graph can be found in Figure 8.

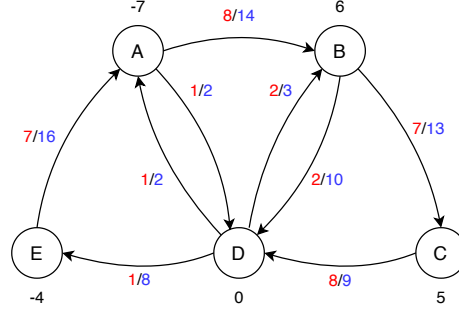


Figure 8: Formulation of repositioning as Min Cost Flow problem

Question 10

To find an initial flow, we start by adding a source s , which is connected to the nodes with negative demand (A and E), and a sink t , connected to the nodes with positive demand (B and C). Then we apply Ford-Fulkerson on the modified graph (without considering the costs). The augmenting paths can be found below for each step and are visualized in Figure 9.

- Step 1: Path $sABt$, capacity 6
- Step 2: Path $sEABt$, capacity 4
- Step 3: Path $sABCt$, capacity 1

After 3 iterations of the algorithm, we find out that the maximum flow is equal to 11. The cost of the initial flow is: $z = 7 * 4 + 8 * 11 + 7 * 5 = 151$.

For the next step, we remove the source s and the sink t and re-add the costs to the graph.

To calculate the flow with the minimum cost we apply the Network Simplex algorithm (see Figure 10). In order for us to apply it, we need to have a tree solution. We see that node D does not belong to the initial flow and that is why we add the edge (C, D) with a flow of 0 in order to reach it and thereby get a tree solution. Then, we set node E as the root node and calculate the distances to the rest of the nodes to get the potentials. To do so we follow the edges of the tree solution from the root to each node and add the cost for each forward edge and subtract it for each backward edge. The root node itself gets

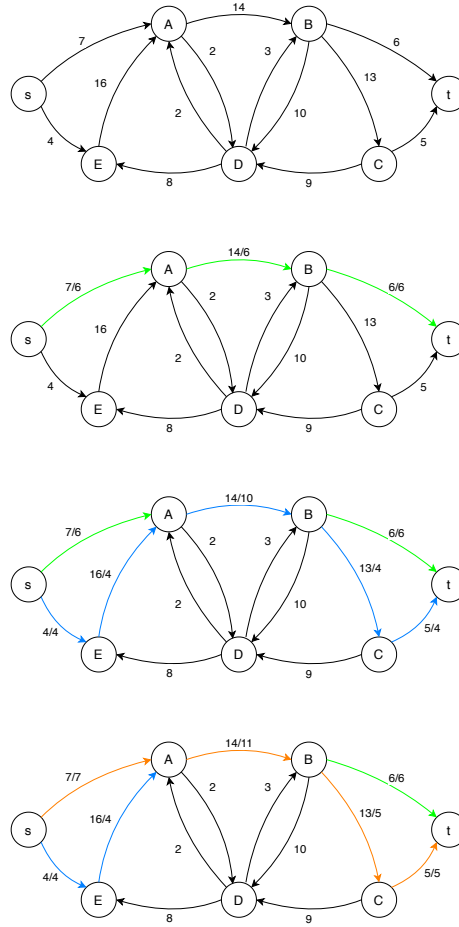


Figure 9: Application of Ford-Fulkerson to modified graph G

a potential of 0. The potentials are indicated in green. From this initial setup, we can now start with the first iteration of the algorithm.

Step 1:

- $L = \{(A, D), (B, D), (D, A), (D, B), (D, E)\}$
 $U = \{\}$
- L:
 reduced cost: $\bar{w}_{AD} = 1 + 7 - 30 = -22$
 reduced cost: $\bar{w}_{BD} = 2 + 15 - 30 = -13$
 reduced cost: $\bar{w}_{DA} = 1 + 30 - 7 = 24$
 reduced cost: $\bar{w}_{DB} = 2 + 30 - 15 = 17$
 reduced cost: $\bar{w}_{DE} = 1 + 30 - 0 = 31$

- edge (A, D) is the L -edge with the most negative reduced cost
- augmenting cycle $c = \{A, D, C, B\}$
- $\theta = \min \{2, 0, 5, 9\} = 0$ can be sent through c
- cost of the flow $z = 151 + \theta * \bar{w}_{AD} = 151 + 0 * (-22) = 151$
- edge (A, D) is added to the tree solution and edge (D, C) is removed from the tree solution
- node D gets potential 8, according to the new tree solution

Step 2:

- $L = \{(B, D), (C, D), (D, A), (D, B), (D, E)\}$
 $U = \{\}$
- L:
reduced cost: $\bar{w}_{BD} = 2 + 15 - 8 = 9$
reduced cost: $\bar{w}_{CD} = 8 + 22 - 8 = 22$
reduced cost: $\bar{w}_{DA} = 1 + 8 - 7 = 2$
reduced cost: $\bar{w}_{DB} = 2 + 8 - 15 = -5$
reduced cost: $\bar{w}_{DE} = 1 + 8 - 0 = 9$
- edge (D, B) is the L -edge with the most negative reduced cost
- augmenting cycle $c = \{D, B, A\}$
- $\theta = \min \{3, 9, 2\} = 2$ can be sent through c
- cost of the flow $z = 151 + \theta * \bar{w}_{DB} = 151 + 2 * (-5) = 141$
- edge (D, B) is added to the tree solution and edge (A, D) is removed from the tree solution
- node D gets potential 13, according to the new tree solution

Step 3:

- $L = \{(B, D), (C, D), (D, A), (D, E)\}$
 $U = \{(A, D)\}$
- L:
reduced cost: $\bar{w}_{BD} = 2 + 15 - 13 = 4$
reduced cost: $\bar{w}_{CD} = 8 + 22 - 13 = 17$
reduced cost: $\bar{w}_{DA} = 1 + 13 - 7 = 7$
reduced cost: $\bar{w}_{DE} = 1 + 13 - 0 = 14$
- U:
reduced cost: $\bar{w}_{AD} = 1 + 7 - 13 = -5$
- All L -edges have positive reduced costs and all U -edges have negative reduced costs, hence the algorithm terminates and we have found an optimal solution with cost $z = 141$, which is displayed in Figure 11.

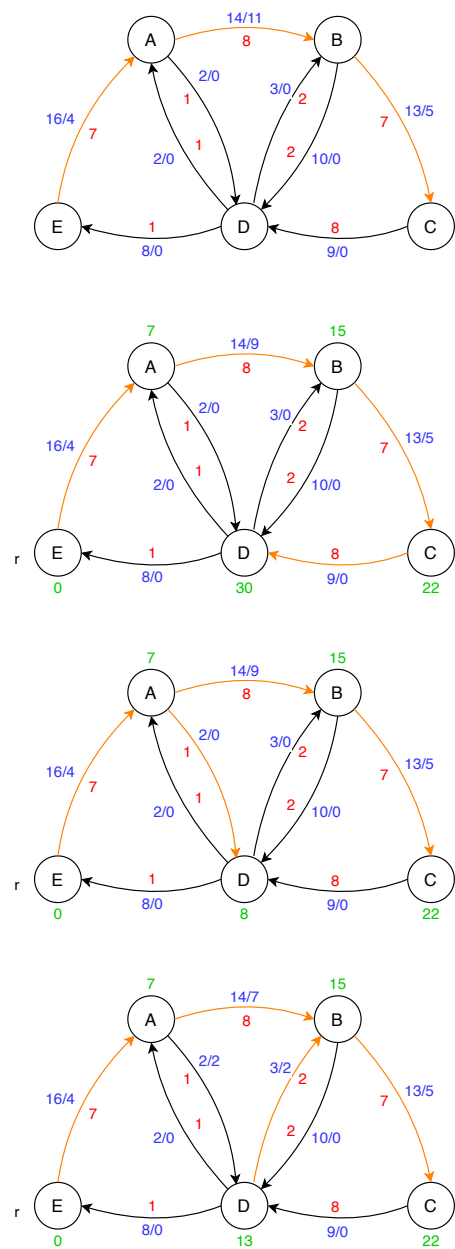


Figure 10: Initial tree solution and application of Network Simplex

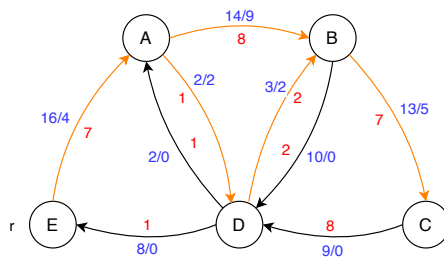


Figure 11: Optimal solution for Min Cost Flow problem

Flowing containers

Question 11

By inserting the given values into the edge formulation, we get the model displayed on the next page. The implementation in Julia can be found in the Appendix. By running it, we get the following solution:

$$z = 319.0$$

$$\begin{aligned} x_{AB}^1 &= 1.0, x_{AB}^2 = 4.0, x_{AB}^3 = 0.0, x_{AD}^1 = 2.0, x_{AD}^2 = 0.0, x_{AD}^3 = 0.0, \\ x_{BC}^1 &= 1.0, x_{BC}^2 = 3.0, x_{BC}^3 = 0.0, x_{BD}^1 = 0.0, x_{BD}^2 = 1.0, x_{BD}^3 = 2.0, \\ x_{CD}^1 &= 1.0, x_{CD}^2 = 3.0, x_{CD}^3 = 0.0, x_{DA}^1 = 0.0, x_{DA}^2 = 0.0, x_{DA}^3 = 0.0, \\ x_{DB}^1 &= 0.0, x_{DB}^2 = 0.0, x_{DB}^3 = 0.0, x_{DE}^1 = 0.0, x_{DE}^2 = 0.0, x_{DE}^3 = 2.0, \\ x_{EA}^1 &= 0.0, x_{EA}^2 = 4.0, x_{EA}^3 = 2.0 \end{aligned}$$

From this, we can conclude the routes for the containers displayed in Table 1.

Commodity k	Container	Route
1	1	$A \rightarrow D$
1	2	$A \rightarrow D$
1	3	$A \rightarrow B \rightarrow C \rightarrow D$
2	1	$E \rightarrow A \rightarrow B \rightarrow C \rightarrow D$
2	2	$E \rightarrow A \rightarrow B \rightarrow C \rightarrow D$
2	3	$E \rightarrow A \rightarrow B \rightarrow C \rightarrow D$
2	4	$E \rightarrow A \rightarrow B \rightarrow D$
3	1	$B \rightarrow D \rightarrow E \rightarrow A$
3	2	$B \rightarrow D \rightarrow E \rightarrow A$

Table 1: Routes of the containers in the IP-solution

$$\min 8x_{AB}^1 + 1x_{AD}^1 + 7x_{BC}^1 + 2x_{BD}^1 + 8x_{CD}^1 \quad (1)$$

$c^*w^*x_{ij}^k$

K s t d w
1 A D 3 1
2 E D 4 2
3 B A 2 4

B1, C1, E1 --> MONO
GIA TA NODES POU
DEN ANOIKOUN STO K

OSA MPAINOUN - OSA
VGAINOUN = 0

$$\text{s.t } B_1 : x_{DB}^1 + x_{AB}^1 - x_{BD}^1 - x_{BC}^1 = 0 \quad (2)$$

$$\underline{C_1} : x_{BC}^1 - x_{CD}^1 = 0 \quad (3)$$

$$\underline{E_1} : x_{DE}^1 - x_{EA}^1 = 0 \quad (4)$$

$$A_2 : x_{EA}^2 + x_{DA}^2 - x_{AB}^2 - x_{AD}^2 = 0 \quad (5)$$

$$\underline{B_2} : x_{DB}^2 + x_{AB}^2 - x_{BD}^2 - x_{BC}^2 = 0 \quad (6)$$

$$\underline{C_2} : x_{BC}^2 - x_{CD}^2 = 0 \quad (7)$$

$$\underline{C_3} : x_{BC}^3 - x_{CD}^3 = 0 \quad (8)$$

$$\underline{D_3} : x_{CD}^3 + x_{BD}^3 + x_{AD}^3 - x_{DB}^3 - x_{DA}^3 - x_{DE}^3 = 0 \quad (9)$$

$$\underline{E_3} : x_{DE}^3 - x_{EA}^3 = 0 \quad (10)$$

$$A_1 : x_{EA}^1 + x_{DA}^1 - x_{AB}^1 - x_{AD}^1 = -3 \quad (11)$$

$$\text{S } E_2 : x_{DE}^2 - x_{EA}^2 = -4 \quad (12)$$

$$B_3 : x_{DB}^3 + x_{AB}^3 - x_{BD}^3 - x_{BC}^3 = -2 \quad (13)$$

$$D_1 : x_{CD}^1 + x_{BD}^1 + x_{AD}^1 - x_{DB}^1 - x_{DA}^1 - x_{DE}^1 = 3 \quad (14)$$

$$\text{t } D_2 : x_{CD}^2 + x_{BD}^2 + x_{AD}^2 - x_{DB}^2 - x_{DA}^2 - x_{DE}^2 = 4 \quad (15)$$

$$A_3 : x_{EA}^3 + x_{DA}^3 - x_{AB}^3 - x_{AD}^3 = 2 \quad (16)$$

$$AB : 1x_{AB}^1 + 2x_{AB}^2 + 4x_{AB}^3 \leq 14 \quad (17)$$

$$AD : 1x_{AD}^1 + 2x_{AD}^2 + 4x_{AD}^3 \leq 2 \quad (18)$$

$$BC : 1x_{BC}^1 + 2x_{BC}^2 + 4x_{BC}^3 \leq 13 \quad (19)$$

$$BD : 1x_{BD}^1 + 2x_{BD}^2 + 4x_{BD}^3 \leq 10 \quad (20)$$

$$CD : 1x_{CD}^1 + 2x_{CD}^2 + 4x_{CD}^3 \leq 9 \quad (21)$$

$$DA : 1x_{DA}^1 + 2x_{DA}^2 + 4x_{DA}^3 \leq 2 \quad (22)$$

$$DB : 1x_{DB}^1 + 2x_{DB}^2 + 4x_{DB}^3 \leq 3 \quad (23)$$

$$DE : 1x_{DE}^1 + 2x_{DE}^2 + 4x_{DE}^3 \leq 8 \quad (24)$$

$$EA : 1x_{EA}^1 + 2x_{EA}^2 + 4x_{EA}^3 \leq 16 \quad (25)$$

$$\text{integer } x_{AB}^1 x_{AD}^1 x_{BC}^1 x_{BD}^1 x_{CD}^1 x_{DA}^1 x_{DB}^1 x_{DE}^1 x_{EA}^1 \quad (26)$$

$$x_{AB}^2 x_{AD}^2 x_{BC}^2 x_{BD}^2 x_{CD}^2 x_{DA}^2 x_{DB}^2 x_{DE}^2 x_{EA}^2 \quad (27)$$

$$x_{AB}^3 x_{AD}^3 x_{BC}^3 x_{BD}^3 x_{CD}^3 x_{DA}^3 x_{DB}^3 x_{DE}^3 x_{EA}^3 \quad (28)$$

oles oi akmes = 9 synolika

capacities

Question 12

For solving the edge formulation as an LP-problem, we leave out the constraint that the x_{ij} variables have to be integer. The implementation of the LP-problem in Julia can be found in the Appendix. It gives the following solution:

$$z = 305.0$$

$$\begin{aligned} x_{AB}^1 &= 1.0, x_{AB}^2 = 4.0, x_{AB}^3 = 0.0, x_{AD}^1 = 2.0, x_{AD}^2 = 0.0, x_{AD}^3 = 0.0, \\ x_{BC}^1 &= 0.0, x_{BC}^2 = 0.0, x_{BC}^3 = 1.75, x_{BD}^1 = 1.0, x_{BD}^2 = 4.0, x_{BD}^3 = 0.25, \\ x_{CD}^1 &= 0.0, x_{CD}^2 = 0.0, x_{CD}^3 = 1.75, x_{DA}^1 = 0.0, x_{DA}^2 = 0.0, x_{DA}^3 = 0.5, \\ x_{DB}^1 &= 0.0, x_{DB}^2 = 0.0, x_{DB}^3 = 0.0, x_{DE}^1 = 0.0, x_{DE}^2 = 0.0, x_{DE}^3 = 1.5, \\ x_{EA}^1 &= 0.0, x_{EA}^2 = 4.0, x_{EA}^3 = 1.5 \end{aligned}$$

In Table 2, the routes for the containers are listed. Since some of the solution values are fractional, in this case one of the containers for commodity 3 has to be split up into three parts (which means that the solution is not feasible for the original (IP) problem).

Commodity k	Container	Route
1	1	$A \rightarrow D$
1	2	$A \rightarrow D$
1	3	$A \rightarrow B \rightarrow D$
2	1	$E \rightarrow A \rightarrow B \rightarrow D$
2	2	$E \rightarrow A \rightarrow B \rightarrow D$
2	3	$E \rightarrow A \rightarrow B \rightarrow D$
2	4	$E \rightarrow A \rightarrow B \rightarrow D$
3	1	$B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$
3	2 (0.5 %)	$B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$
3	2 (0.25 %)	$B \rightarrow C \rightarrow D \rightarrow A$
3	2 (0.25 %)	$B \rightarrow D \rightarrow A$

Table 2: Routes of the containers in the LP-solution

Question 13

The LP solution gives a lower bound of 305, whereas the IP solution is 319. This means we have a gap of 4.39 %, which is quite good.

Question 14

In Table 3, all possible simple routes for every commodity are listed, together with their costs.

Route number	Commodity k	Route	Cost for w_k TEU
1	1	$A \rightarrow D$	1
2	1	$A \rightarrow B \rightarrow D$	$8+2 = 10$
3	1	$A \rightarrow B \rightarrow C \rightarrow D$	$8+7+8 = 23$
4	2	$E \rightarrow A \rightarrow D$	$2*(7+1) = 16$
5	2	$E \rightarrow A \rightarrow B \rightarrow D$	$2*(7+8+2) = 34$
6	2	$E \rightarrow A \rightarrow B \rightarrow C \rightarrow D$	$2*(7+8+7+8) = 60$
7	3	$B \rightarrow D \rightarrow E \rightarrow A$	$4*(2+1+7) = 40$
8	3	$B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$	$4*(7+8+1+7) = 92$

Table 3: Possible simple routes

Question 15

By inserting the paths and costs found in Question 14 into the path formulation, we get the following model for our problem:

$$\begin{aligned}
 \min \quad & \underline{1}x_1 + \underline{10}x_2 + \underline{23}x_3 + \underline{16}x_4 + \underline{34}x_5 + \underline{60}x_6 + \underline{40}x_7 + \underline{92}x_8 & (29) \\
 \text{s.t.} \quad & \underline{AB} : \underline{1}x_2 + \underline{1}x_3 + \underline{2}x_5 + \underline{2}x_6 & \leq 14 & (30) \\
 & AD : 1x_1 + 2x_4 & \leq 2 & (31) \\
 & BC : 1x_3 + 2x_6 + 4x_8 & \leq 13 & (32) \\
 & BD : 1x_2 + 2x_5 + 4x_7 & \leq 10 & (33) \\
 & CD : 1x_3 + 2x_6 + 4x_8 & \leq 9 & (34) \\
 & DA : 0 & \leq 2 & (35) \\
 & DB : 0 & \leq 3 & (36) \\
 & DE : 4x_7 + 4x_8 & \leq 8 & (37) \\
 & EA : 2x_4 + 2x_5 + 2x_6 + 4x_7 + 4x_8 & \leq 16 & (38) \\
 & K1 : x_1 + x_2 + x_3 & \geq 3 & (39) \\
 & K2 : x_4 + x_5 + x_6 & \geq 4 & (40) \\
 & K3 : x_7 + x_8 & \geq 2 & (41) \\
 \text{integer} \quad & x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6 \ x_7 \ x_8 & & (42)
 \end{aligned}$$

g edges (handwritten blue text next to the first column of constraints)

capacities (handwritten orange text next to the right column of constraints)

d (handwritten green text next to the right column of constraints)

Since the inequalities (35) and (36) are always true, we can leave them out when solving the problem.

Question 16

We have implemented the model from Question 15 in Julia, defining the x_i variables as integers, since we want to solve the problem to IP-optimality. The corresponding code can be found in the Appendix. Solving the problem gives the following solution:

$z = 319$
 $x_1 = 2$
 $x_2 = 0$
 $x_3 = 1$
 $x_4 = 0$
 $x_5 = 3$
 $x_6 = 1$
 $x_7 = 1$
 $x_8 = 1$

Question 17

To solve the problem to LP-optimality, we reuse the code from Question 16, with the only difference that now the variables are not defined as integers. The modified code is enclosed in the Appendix. This version of the model gives the following solution:

$z = 319$
 $x_1 = 2$
 $x_2 = 1$
 $x_3 = 0$
 $x_4 = 0$
 $x_5 = 4$
 $x_6 = 0$
 $x_7 = 0.25$
 $x_8 = 1.75$

Question 18

By comparing the IP and the LP solution, we see that the objective values are exactly the same. Therefore, we can say that the LP solution provides the best possible lower bound.

Question 19

To show that any feasible solution to the LP-relaxed path formulation is also a feasible solution to the LP-relaxed edge formulation, we first assume that $x = (x_r^k, k \in K, r \in R_k)$ is a feasible solution to the LP-relaxed path formulation. Now we have to show that x is also a feasible solution to the LP-relaxed edge formulation. Before we can show the feasibility of x , we first have to transform it to a solution of the edge formulation: For every edge $(i, j) \in E$, we define x_{ij}^k as $\sum_{r \in R_k^{ij}} x_r^k$, where R_k^{ij} is defined as the set of routes for commodity k that pass through the edge (i, j) . To show the feasibility of this solution for the edge formulation, we go through the constraints in the edge formulation and show that they must be satisfied given that x is a solution to the path formulation. In

the following, the equation numbers reference those in the project description. The flow conservation constraint (2) must be satisfied due to the fact that for our original x we have considered routes, within which the amount of flow going into a node always equals the amount of flow going out of it. Constraints (3) and (4) are ensured by constraint (9): In constraint (9), the left hand side never gets larger than d_k due to the fact that it is a minimization problem. Furthermore, since we have considered routes for our original x , at the starting points of the routes there is only outgoing flow and on the ending points only ingoing flow (if we consider the routes separately). Constraints (5) and (the relaxed version of) (6) follow directly from constraints (9) and (the relaxed version of) (10), respectively, if we take into account the transformation we have made in the beginning.

The only thing that is left to show is that there are instances for which the path formulation gives tighter bounds than the edge formulation. But in fact, the MeditLine instance given in this project provides an example for exactly this case: As seen in Question 12, the LP-relaxed edge formulation gives a lower bound of 305 for the objective value of the IP problem, whereas the LP-relaxed path formulation results in 319, which is a tighter lower bound (and actually coincides with the objective value of the IP solution).

Question 20

To calculate the reduced cost of a route r , we can use a similar formula as for the reduced costs for a splittable multicommodity flow problem, with only one difference: We have to multiply the sum $\sum_{(i,j) \in r} (c_{ij} - y_{ij})$ by w_k , since we send w_k TEU and not just one unit through each edge used by a route for commodity k . Consequently, we can calculate the reduced costs $\bar{c}_r^k$ as follows:

$$\bar{c}_r^k = w_k \sum_{(i,j) \in r} (c_{ij} - y_{ij}) - \bar{y}_k \quad (43)$$

Applying this formula, we get the reduced costs for the routes in $R_1 \cup R_2 \cup R_3$ listed in Table 4.

Route no.	Comm. k	Route	$w_k \sum_{(i,j) \in r} (c_{ij} - y_{ij})$	$\bar{c}_r^k$
1	1	$A \rightarrow D$	1	-22
2	1	$A \rightarrow B \rightarrow D$	$8 + (2 + 13) = 23$	0
3	1	$A \rightarrow B \rightarrow C \rightarrow D$	$8 + 7 + 8 = 23$	0
4	2	$E \rightarrow A \rightarrow D$	$2^*(7 + 1) = 16$	-44
5	2	$E \rightarrow A \rightarrow B \rightarrow D$	$2^*(7 + 8 + (2 + 13)) = 60$	0
6	2	$E \rightarrow A \rightarrow B \rightarrow C \rightarrow D$	$2^*(7 + 8 + 7 + 8) = 60$	0
7	3	$B \rightarrow D \rightarrow E \rightarrow A$	$4^*((2 + 13) + 1 + 7) = 92$	0
8	3	$B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$	$4^*(7 + 8 + 1 + 7) = 92$	0

Table 4: Reduced costs for all possible simple routes

From Table 4, we see that route 4 ($E \rightarrow A \rightarrow D$) has the most negative reduced cost with -44. That means that this route should be added to the master problem in the next iteration.

Question 21

In Question 20, we used the formula $\bar{c}_r^k = w_k \sum_{(i,j) \in r} (c_{ij} - y_{ij}) - \bar{y}_k$ to compute the reduced costs. For each commodity k , we see that $\bar{y}_k$ is a constant. Therefore, to get the smallest reduced cost we need to minimize the expression: $w_k \sum_{(i,j) \in r} (c_{ij} - y_{ij})$. This can be done by solving the shortest-path problem from source s_k to terminal t_k with edge costs $w_k(c_{ij} - y_{ij})$. The edge costs can never be negative, because in order for that to happen, there needs to be a positive y_{ij} variable (which is larger than c_{ij}). However, for minimization problems, the dual variables corresponding to " $\leq$ " constraints must always be non-positive, hence the y_{ij} variables will always be negative.

First, we modify the costs, and after that, we will be ready to apply the code we implemented in Question 2. The networks on which we apply the algorithm are shown in Figure 12.

Applying the shortest path algorithm and subtracting the dual variables $\bar{y}_k$ gives the minimum reduced costs for each commodity, which are listed in Table 5. We see that route 4 has the most negative reduced cost (-44), which coincides with our result in Question 20. This means that route 4 should be added to the master problem.

Route no.	Comm. k	Route	SPP solution	$\bar{c}_r^k$
1	1	$A \rightarrow D$	1	-22
4	2	$E \rightarrow A \rightarrow D$	16	-44
7	3	$B \rightarrow D \rightarrow E \rightarrow A$	92	0

Table 5: Reduced costs for shortest paths in iteration 1

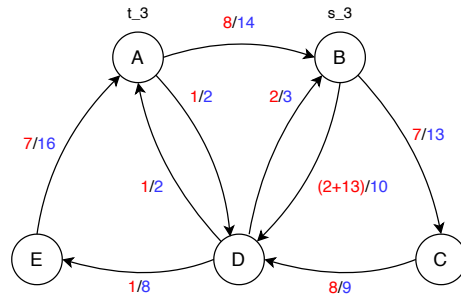
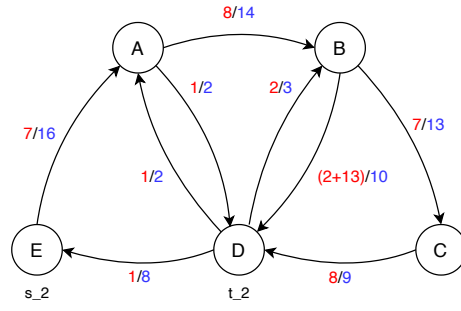
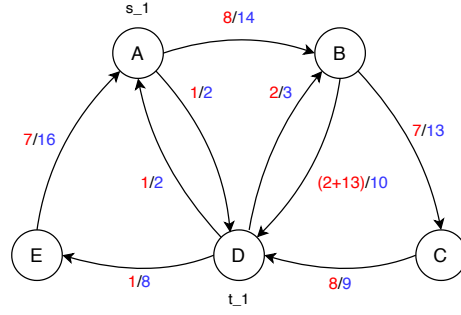


Figure 12: Shortest path problem for each commodity

Question 22

After adding path 4 to the master problem, we get the following updated problem:

$$\min 10x_2 + 23x_3 + 16x_4 + 34x_5 + 92x_8 \quad (44)$$

$$\text{s.t } AB : 1x_2 + 1x_3 + 2x_5 \leq 14 \quad (45)$$

$$AD : 2x_4 \leq 2 \quad (46)$$

$$BC : 1x_3 + 4x_8 \leq 13 \quad (47)$$

$$BD : 1x_2 + 2x_5 \leq 10 \quad (48)$$

$$CD : 1x_3 + 4x_8 \leq 9 \quad (49)$$

$$DA : 0 \leq 2 \quad (50)$$

$$DB : 0 \leq 3 \quad (51)$$

$$DE : 4x_8 \leq 8 \quad (52)$$

$$EA : 2x_4 + 2x_5 + 4x_8 \leq 16 \quad (53)$$

$$K1 : x_2 + x_3 \geq 3 \quad (54)$$

$$K2 : x_4 + x_5 \geq 4 \quad (55)$$

$$K3 : x_8 \geq 2 \quad (56)$$

$$x_2, x_3, x_4, x_5, x_8 \geq 0 \quad (57)$$

For implementing the model in Julia, we can reuse the code from Question 17 and just add constraints for making the columns that are currently excluded equal to 0 ($x_i \leq 0$). The code can be found in the Appendix. By applying the code, we get the following solution:

The objective value is:

$$z = 332$$

The solution is:

$$x_1 = 0$$

$$x_2 = 3$$

$$x_3 = 0$$

$$x_4 = 1$$

$$x_5 = 3$$

$$x_6 = 0$$

$$x_7 = 0$$

$$x_8 = 2$$

Dual variables: $y_{AD} = -9.0$ and all other $y_{ij} = 0$

$$\bar{y}_1 = 10, \bar{y}_2 = 34, \bar{y}_3 = 92$$

Question 23

Iteration 2

By modifying the edge costs, then applying the shortest path algorithm and subtracting the dual variables from the results, we get the reduced costs given in Table 6.

Route no.	Comm. k	Route	SPP solution	$\bar{c}_r^k$
1	1	$A \rightarrow D$	10	0
4	2	$E \rightarrow A \rightarrow D$	34	0
7	3	$B \rightarrow D \rightarrow E \rightarrow A$	92	-52

Table 6: Reduced costs for shortest paths in iteration 2

Route 7 is the only one that has negative reduced cost, so we add it to the master problem:

$$\min 10x_2 + 23x_3 + 16x_4 + 34x_5 + 40x_7 + 92x_8 \quad (58)$$

$$\text{s.t } AB : 1x_2 + 1x_3 + 2x_5 \leq 14 \quad (59)$$

$$AD : 2x_4 \leq 2 \quad (60)$$

$$BC : 1x_3 + 4x_8 \leq 13 \quad (61)$$

$$BD : 1x_2 + 2x_5 + 4x_7 \leq 10 \quad (62)$$

$$CD : 1x_3 + 4x_8 \leq 9 \quad (63)$$

$$DA : 0 \leq 2 \quad (64)$$

$$DB : 0 \leq 3 \quad (65)$$

$$DE : 4x_7 + 4x_8 \leq 8 \quad (66)$$

$$EA : 2x_4 + 2x_5 + 4x_7 + 4x_8 \leq 16 \quad (67)$$

$$K1 : x_2 + x_3 \geq 3 \quad (68)$$

$$K2 : x_4 + x_5 \geq 4 \quad (69)$$

$$K3 : x_7 + x_8 \geq 2 \quad (70)$$

$$x_2, x_3, x_4, x_5, x_7, x_8 \geq 0 \quad (71)$$

Reusing the code from Question 22, with the only difference that we comment out the line $x_7 \leq 0$, we get the following solution for the current master problem:

The objective value is:

$$z = 319$$

The solution is:

$$x_1 = 0$$

$$x_2 = 3.0$$

$x_3 = 0.0$
 $x_4 = 1.0$
 $x_5 = 3.0$
 $x_6 = 0.0$
 $x_7 = 0.25$
 $x_8 = 1.75$

Dual variables:

$y_{AD} = -22$, $y_{BD} = -13$ and all other $y_{ij} = 0$
 $y_1 = 23$, $y_2 = 60$, $y_3 = 92$

Iteration 3

By modifying the edge costs, then applying the shortest path algorithm and subtracting the dual variables from the results, we get the reduced costs given in Table 7.

Route no.	Comm. k	Route	SPP solution	$\bar{c}_r^k$
1	1	$A \rightarrow D$	23	0
4	2	$E \rightarrow A \rightarrow D$	60	0
7	3	$B \rightarrow D \rightarrow E \rightarrow A$	92	0

Table 7: Reduced costs for shortest paths in iteration 3

As we can see from the table, there are no more routes with negative reduced cost, which means that the algorithm terminates and we have found an optimal solution, with objective value 319.

Question 24

In the column generation approach, we ended up adding two routes (route 4 and route 7) to the master problem. This means that there are only two out of eight routes which are still not included in the master problem in the end. Consequently, for this particular problem, we can say that the column generation approach did not bring significant decrease in running and modeling time, whereas for larger problems in practice, it will result in a faster calculation of the solution.

Appendix

Question 2

```
import java.io.File;
import java.util.Scanner;
import java.util.Arrays;

public class BellmanFord {

    public static void main(String[] args) throws Exception {

        //scanning of text file
        //insert path for text file
        File file = new File("C:\\Users\\spfre\\Desktop\\
            Bellmanfordtest.txt");
        Scanner sc = new Scanner(file);

        int n = sc.nextInt();
        int m = sc.nextInt();
        int s = sc.nextInt()-1;
        int t = sc.nextInt()-1;
        int w = sc.nextInt();
        String Output0 = String.format("n=%d, m=%d, s=%d
            , t=%d, w=%d", n, m, s+1, t+1, w);
            //System.out.println(Output0)
            ;

        //create matrix for edges and respective edge info
        int [][] edges = new int[m][4];

        //check how many of these edges have sufficient
            capacity
        int reali = 0;

        for (int i = 0; i < m; i++) {
            sc.nextLine();
            int a = sc.nextInt();
            int b = sc.nextInt();
            int c = sc.nextInt();
            int u = sc.nextInt();
            String Output1 = String.format("a=%d, b=%d, c=
                =%d, u=%d", a, b, c, u);
                //System.out.println(Output1);

            if (u >= w ) {
                edges[reali][0] = a-1;
                edges[reali][1] = b-1;
                edges[reali][2] = w*c;
                edges[reali][3] = u;
            }
        }
    }
}
```

```

        reali++;
    }
}
sc.close();

//System.out.println("Real i =
                        " + reali);

//initial setup: infinite distance and no predecessor
int [][] nodes = new int[n][2];
for (int p = 0; p < n; p++) {
    nodes[p][0] = 1000;
    nodes[p][1] = -1;
}
nodes[s][0] = 0;

//actual iterations: check if distances can be
improved
for (int k = 0; k < n-1; k++) {
    for (int l = 0; l < reali; l++) {
        int a = edges[l][0];
        int b = edges[l][1];
        int c = edges[l][2];
        if((nodes[a][0] + c) < nodes[b][0]) {
            nodes[b][0] = nodes[a][0] + c;
            nodes[b][1] = a;
            //System.out.println("Did
                                something");
        }
        //System.out.println("Did nothing
                                ");
    }
}

//check for negative cycles
for (int l = 0; l < reali; l++) {
    int a = edges[l][0];
    int b = edges[l][1];
    int c = edges[l][2];
    if((nodes[a][0] + c) < nodes[b][0]) {
        System.out.println("Error: negative cycle");
    }
}

for (int ab = 0; ab < m; ab++) {
    String Output5 = String.format("edge %d, a=%d,
    b=%d, c=%d, u=%d", ab+1, edges[ab]
    [0]+1, edges[ab][1]+1, edges[ab][2], edges[ab]
    [3]);
    //System.out.println(Output5);
}

```

```

    for (int ab = 0; ab < n; ab++) {
        String Output6 = String.format("node=%d, cost=%d, previous=%d", ab+1, nodes[ab][0], nodes[ab][1]+1);
        //System.out.println(Output6);
    }

    //calculate and print cost, reverse path, starting and endpoint
    int i_c = t;
    int cost = nodes[t][0];
    String Output = String.format("Cost: %d\nPath (backwards): %d", cost, t+1);
    System.out.print(Output);
    while (i_c != s) {
        int i_n = nodes[i_c][1];
        int i_o = i_n + 1;
        System.out.print("  " + i_o);
        i_c = i_n;
    }
}
}
}

```

Question 11

using JuMP, GLPK

```
model = Model(GLPK.Optimizer)
```

#IP FORMULATION

```

@variable(model, xAB_1 >= 0, Int)
@variable(model, xAD_1 >= 0, Int)
@variable(model, xBC_1 >= 0, Int)
@variable(model, xBD_1 >= 0, Int)
@variable(model, xCD_1 >= 0, Int)
@variable(model, xDA_1 >= 0, Int)
@variable(model, xDB_1 >= 0, Int)
@variable(model, xDE_1 >= 0, Int)
@variable(model, xEA_1 >= 0, Int)
@variable(model, xAB_2 >= 0, Int)
@variable(model, xAD_2 >= 0, Int)
@variable(model, xBC_2 >= 0, Int)
@variable(model, xBD_2 >= 0, Int)
@variable(model, xCD_2 >= 0, Int)
@variable(model, xDA_2 >= 0, Int)
@variable(model, xDB_2 >= 0, Int)

```

```

@variable(model, xDE_2 >= 0, Int)
@variable(model, xEA_2 >= 0, Int)
@variable(model, xAB_3 >= 0, Int)
@variable(model, xAD_3 >= 0, Int)
@variable(model, xBC_3 >= 0, Int)
@variable(model, xBD_3 >= 0, Int)
@variable(model, xCD_3 >= 0, Int)
@variable(model, xDA_3 >= 0, Int)
@variable(model, xDB_3 >= 0, Int)
@variable(model, xDE_3 >= 0, Int)
@variable(model, xEA_3 >= 0, Int)

@objective(model, Min, 8*xAB_1 + 1*xAD_1 + 7*xBC_1 + 2*xBD_1
+ 8*xCD_1 + 1*xDA_1 + 2*xDB_1 + 1*xDE_1 + 7*xEA_1
+ 16*xAB_2 + 2*xAD_2 + 14*xBC_2 + 4*xBD_2
+ 16*xCD_2 + 2*xDA_2 + 4*xDB_2 + 2*xDE_2 + 14*xEA_2
+ 32*xAB_3 + 4*xAD_3 + 28*xBC_3 + 8*xBD_3
+ 32*xCD_3 + 4*xDA_3 + 8*xDB_3 + 4*xDE_3 + 28*xEA_3)

@constraint(model, xDB_1 + xAB_1 - xBD_1 - xBC_1 == 0)
@constraint(model, xBC_1 - xCD_1 == 0)
@constraint(model, xDE_1 - xEA_1 == 0)
@constraint(model, xEA_2 + xDA_2 - xAB_2 - xAD_2 == 0)
@constraint(model, xDB_2 + xAB_2 - xBD_2 - xBC_2 == 0)
@constraint(model, xBC_2 - xCD_2 == 0)
@constraint(model, xBC_3 - xCD_3 == 0)
@constraint(model, xCD_3 + xBD_3 + xAD_3 - xDB_3 - xDA_3 - xDE_3 == 0)
@constraint(model, xDE_3 - xEA_3 == 0)

@constraint(model, xEA_1 + xDA_1 - xAB_1 - xAD_1 == -3)
@constraint(model, xDE_2 - xEA_2 == -4)
@constraint(model, xDB_3 + xAB_3 - xBD_3 - xBC_3 == -2)
@constraint(model, xCD_1 + xBD_1 + xAD_1 - xDB_1 - xDA_1 - xDE_1 == 3)
@constraint(model, xCD_2 + xBD_2 + xAD_2 - xDB_2 - xDA_2 - xDE_2 == 4)
@constraint(model, xEA_3 + xDA_3 - xAB_3 - xAD_3 == 2)

@constraint(model, 1*xAB_1 + 2*xAB_2 + 4*xAB_3 <= 14)
@constraint(model, 1*xAD_1 + 2*xAD_2 + 4*xAD_3 <= 2)
@constraint(model, 1*xBC_1 + 2*xBC_2 + 4*xBC_3 <= 13)
@constraint(model, 1*xBD_1 + 2*xBD_2 + 4*xBD_3 <= 10)
@constraint(model, 1*xCD_1 + 2*xCD_2 + 4*xCD_3 <= 9)
@constraint(model, 1*xDA_1 + 2*xDA_2 + 4*xDA_3 <= 2)
@constraint(model, 1*xDB_1 + 2*xDB_2 + 4*xDB_3 <= 3)
@constraint(model, 1*xDE_1 + 2*xDE_2 + 4*xDE_3 <= 8)
@constraint(model, 1*xEA_1 + 2*xEA_2 + 4*xEA_3 <= 16)

```

```

JuMP.optimize!(model)

```

```
println("The objective value is: \nz = ", JuMP.objective_value(model))

println("The solution is: ")
println("xAB_1 = ", JuMP.value(xAB_1))
println("xAB_2 = ", JuMP.value(xAB_2))
println("xAB_3 = ", JuMP.value(xAB_3))
println("xAD_1 = ", JuMP.value(xAD_1))
println("xAD_2 = ", JuMP.value(xAD_2))
println("xAD_3 = ", JuMP.value(xAD_3))
println("xBC_1 = ", JuMP.value(xBC_1))
println("xBC_2 = ", JuMP.value(xBC_2))
println("xBC_3 = ", JuMP.value(xBC_3))
println("xBD_1 = ", JuMP.value(xBD_1))
println("xBD_2 = ", JuMP.value(xBD_2))
println("xBD_3 = ", JuMP.value(xBD_3))
println("xCD_1 = ", JuMP.value(xCD_1))
println("xCD_2 = ", JuMP.value(xCD_2))
println("xCD_3 = ", JuMP.value(xCD_3))
println("xDA_1 = ", JuMP.value(xDA_1))
println("xDA_2 = ", JuMP.value(xDA_2))
println("xDA_3 = ", JuMP.value(xDA_3))
println("xDB_1 = ", JuMP.value(xDB_1))
println("xDB_2 = ", JuMP.value(xDB_2))
println("xDB_3 = ", JuMP.value(xDB_3))
println("xDE_1 = ", JuMP.value(xDE_1))
println("xDE_2 = ", JuMP.value(xDE_2))
println("xDE_3 = ", JuMP.value(xDE_3))
println("xEA_1 = ", JuMP.value(xEA_1))
println("xEA_2 = ", JuMP.value(xEA_2))
println("xEA_3 = ", JuMP.value(xEA_3))
```

Question 12

using JuMP, GLPK

```
model = Model(GLPK.Optimizer)
```

#LP FORMULATION

```
@variable(model, xAB_1 >= 0)
@variable(model, xAD_1 >= 0)
@variable(model, xBC_1 >= 0)
@variable(model, xBD_1 >= 0)
@variable(model, xCD_1 >= 0)
@variable(model, xDA_1 >= 0)
@variable(model, xDB_1 >= 0)
@variable(model, xDE_1 >= 0)
@variable(model, xEA_1 >= 0)
```

```

@variable(model, xAB_2 >= 0)
@variable(model, xAD_2 >= 0)
@variable(model, xBC_2 >= 0)
@variable(model, xBD_2 >= 0)
@variable(model, xCD_2 >= 0)
@variable(model, xDA_2 >= 0)
@variable(model, xDB_2 >= 0)
@variable(model, xDE_2 >= 0)
@variable(model, xEA_2 >= 0)
@variable(model, xAB_3 >= 0)
@variable(model, xAD_3 >= 0)
@variable(model, xBC_3 >= 0)
@variable(model, xBD_3 >= 0)
@variable(model, xCD_3 >= 0)
@variable(model, xDA_3 >= 0)
@variable(model, xDB_3 >= 0)
@variable(model, xDE_3 >= 0)
@variable(model, xEA_3 >= 0)

@objective(model, Min, 8*xAB_1 + 1*xAD_1 + 7*xBC_1 + 2*xBD_1
+ 8*xCD_1 + 1*xDA_1 + 2*xDB_1 + 1*xDE_1 + 7*xEA_1
+ 16*xAB_2 + 2*xAD_2 + 14*xBC_2 + 4*xBD_2
+ 16*xCD_2 + 2*xDA_2 + 4*xDB_2 + 2*xDE_2 + 14*xEA_2
+ 32*xAB_3 + 4*xAD_3 + 28*xBC_3 + 8*xBD_3
+ 32*xCD_3 + 4*xDA_3 + 8*xDB_3 + 4*xDE_3 + 28*xEA_3)

@constraint(model, xDB_1 + xAB_1 - xBD_1 - xBC_1 == 0)
@constraint(model, xBC_1 - xCD_1 == 0)
@constraint(model, xDE_1 - xEA_1 == 0)
@constraint(model, xEA_2 + xDA_2 - xAB_2 - xAD_2 == 0)
@constraint(model, xDB_2 + xAB_2 - xBD_2 - xBC_2 == 0)
@constraint(model, xBC_2 - xCD_2 == 0)
@constraint(model, xBC_3 - xCD_3 == 0)
@constraint(model, xCD_3 + xBD_3 + xAD_3 - xDB_3 - xDA_3 - xDE_3 == 0)
@constraint(model, xDE_3 - xEA_3 == 0)

@constraint(model, xEA_1 + xDA_1 - xAB_1 - xAD_1 == -3)
@constraint(model, xDE_2 - xEA_2 == -4)
@constraint(model, xDB_3 + xAB_3 - xBD_3 - xBC_3 == -2)
@constraint(model, xCD_1 + xBD_1 + xAD_1 - xDB_1 - xDA_1 - xDE_1 == 3)
@constraint(model, xCD_2 + xBD_2 + xAD_2 - xDB_2 - xDA_2 - xDE_2 == 4)
@constraint(model, xEA_3 + xDA_3 - xAB_3 - xAD_3 == 2)

@constraint(model, 1*xAB_1 + 2*xAB_2 + 4*xAB_3 <= 14)
@constraint(model, 1*xAD_1 + 2*xAD_2 + 4*xAD_3 <= 2)
@constraint(model, 1*xBC_1 + 2*xBC_2 + 4*xBC_3 <= 13)
@constraint(model, 1*xBD_1 + 2*xBD_2 + 4*xBD_3 <= 10)
@constraint(model, 1*xCD_1 + 2*xCD_2 + 4*xCD_3 <= 9)

```

```

@constraint(model, 1*xDA_1 + 2*xDA_2 + 4*xDA_3 <= 2)
@constraint(model, 1*xDB_1 + 2*xDB_2 + 4*xDB_3 <= 3)
@constraint(model, 1*xDE_1 + 2*xDE_2 + 4*xDE_3 <= 8)
@constraint(model, 1*xEA_1 + 2*xEA_2 + 4*xEA_3 <= 16)

JuMP.optimize!(model)

println("The objective value is: \nz = ", JuMP.objective_value(model))

println("The solution is: ")
println("xAB_1 = ", JuMP.value(xAB_1))
println("xAB_2 = ", JuMP.value(xAB_2))
println("xAB_3 = ", JuMP.value(xAB_3))
println("xAD_1 = ", JuMP.value(xAD_1))
println("xAD_2 = ", JuMP.value(xAD_2))
println("xAD_3 = ", JuMP.value(xAD_3))
println("xBC_1 = ", JuMP.value(xBC_1))
println("xBC_2 = ", JuMP.value(xBC_2))
println("xBC_3 = ", JuMP.value(xBC_3))
println("xBD_1 = ", JuMP.value(xBD_1))
println("xBD_2 = ", JuMP.value(xBD_2))
println("xBD_3 = ", JuMP.value(xBD_3))
println("xCD_1 = ", JuMP.value(xCD_1))
println("xCD_2 = ", JuMP.value(xCD_2))
println("xCD_3 = ", JuMP.value(xCD_3))
println("xDA_1 = ", JuMP.value(xDA_1))
println("xDA_2 = ", JuMP.value(xDA_2))
println("xDA_3 = ", JuMP.value(xDA_3))
println("xDB_1 = ", JuMP.value(xDB_1))
println("xDB_2 = ", JuMP.value(xDB_2))
println("xDB_3 = ", JuMP.value(xDB_3))
println("xDE_1 = ", JuMP.value(xDE_1))
println("xDE_2 = ", JuMP.value(xDE_2))
println("xDE_3 = ", JuMP.value(xDE_3))
println("xEA_1 = ", JuMP.value(xEA_1))
println("xEA_2 = ", JuMP.value(xEA_2))
println("xEA_3 = ", JuMP.value(xEA_3))

```

Question 16

```

using JuMP, GLPK

model = Model{GLPK.Optimizer}

#IP FORMULATION

@variable(model, x1 >= 0, Int)
@variable(model, x2 >= 0, Int)

```

```

@variable(model, x3 >= 0, Int)
@variable(model, x4 >= 0, Int)
@variable(model, x5 >= 0, Int)
@variable(model, x6 >= 0, Int)
@variable(model, x7 >= 0, Int)
@variable(model, x8 >= 0, Int)

@objective(model, Min, 1*x1 + 10*x2 + 23*x3 + 16*x4
                      + 34*x5 + 60*x6 + 40*x7 + 92*x8 )

@constraint(model, 1*x2 + 1*x3 + 2*x5 + 2*x6 <= 14)
@constraint(model, 1*x1 + 2*x4 <= 2)
@constraint(model, 1*x3 + 2*x6 + 4*x8 <= 13)
@constraint(model, 1*x2 + 2*x5 + 4*x7 <= 10)
@constraint(model, 1*x3 + 2*x6 + 4*x8 <= 9)
@constraint(model, 4*x7 + 4*x8 <= 8)
@constraint(model, 2*x4 + 2*x5 + 2*x6 + 4*x7 + 4*x8 <= 16)
@constraint(model, x1 + x2 + x3 >= 3)
@constraint(model, x4 + x5 + x6 >= 4)
@constraint(model, x7 + x8 >= 2)

JuMP.optimize!(model)

println("The objective value is: \nz = ", JuMP.objective_value(model))

println("The solution is: ")
println("x1 = ", JuMP.value(x1))
println("x2 = ", JuMP.value(x2))
println("x3 = ", JuMP.value(x3))
println("x4 = ", JuMP.value(x4))
println("x5 = ", JuMP.value(x5))
println("x6 = ", JuMP.value(x6))
println("x7 = ", JuMP.value(x7))
println("x8 = ", JuMP.value(x8))

```

Question 17

```

using JuMP, GLPK

model = Model{GLPK.Optimizer}

#LP FORMULATION

@variable(model, x1 >= 0)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)
@variable(model, x4 >= 0)
@variable(model, x5 >= 0)

```

```

@variable(model, x6 >= 0)
@variable(model, x7 >= 0)
@variable(model, x8 >= 0)

@objective(model, Min, 1*x1 + 10*x2 + 23*x3 + 16*x4
                      + 34*x5 + 60*x6 + 40*x7 + 92*x8 )

@constraint(model, 1*x2 + 1*x3 + 2*x5 + 2*x6 <= 14)
@constraint(model, 1*x1 + 2*x4 <= 2)
@constraint(model, 1*x3 + 2*x6 + 4*x8 <= 13)
@constraint(model, 1*x2 + 2*x5 + 4*x7 <= 10)
@constraint(model, 1*x3 + 2*x6 + 4*x8 <= 9)
@constraint(model, 4*x7 + 4*x8 <= 8)
@constraint(model, 2*x4 + 2*x5 + 2*x6 + 4*x7 + 4*x8 <= 16)
@constraint(model, x1 + x2 + x3 >= 3)
@constraint(model, x4 + x5 + x6 >= 4)
@constraint(model, x7 + x8 >= 2)

JuMP.optimize!(model)

println("The objective value is: \nz = ", JuMP.objective_value(model))

println("The solution is: ")
println("x1 = ", JuMP.value(x1))
println("x2 = ", JuMP.value(x2))
println("x3 = ", JuMP.value(x3))
println("x4 = ", JuMP.value(x4))
println("x5 = ", JuMP.value(x5))
println("x6 = ", JuMP.value(x6))
println("x7 = ", JuMP.value(x7))
println("x8 = ", JuMP.value(x8))

```

Question 22

```

using JuMP, GLPK

model = Model{GLPK.Optimizer}

#LP FORMULATION

@variable(model, x1 >= 0)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)
@variable(model, x4 >= 0)
@variable(model, x5 >= 0)
@variable(model, x6 >= 0)
@variable(model, x7 >= 0)
@variable(model, x8 >= 0)

```

```

@objective(model, Min, 1*x1 + 10*x2 + 23*x3 + 16*x4
                    + 34*x5 + 60*x6 + 40*x7 + 92*x8 )

@constraint(model, yAB, 1*x2 + 1*x3 + 2*x5 + 2*x6 <= 14)
@constraint(model, yAD, 1*x1 + 2*x4 <= 2)
@constraint(model, yBC, 1*x3 + 2*x6 + 4*x8 <= 13)
@constraint(model, yBD, 1*x2 + 2*x5 + 4*x7 <= 10)
@constraint(model, yCD, 1*x3 + 2*x6 + 4*x8 <= 9)
@constraint(model, yDE, 4*x7 + 4*x8 <= 8)
@constraint(model, yEA, 2*x4 + 2*x5 + 2*x6 + 4*x7 + 4*x8 <= 16)

@constraint(model, y1, x1 + x2 + x3 >= 3)
@constraint(model, y2, x4 + x5 + x6 >= 4)
@constraint(model, y3, x7 + x8 >= 2)

# Column selection (the columns that are used
                    # in the master problem are commented out):
@constraint(model, x1 <= 0)
#@constraint(model, x2 <= 0)
#@constraint(model, x3 <= 0)
#@constraint(model, x4 <= 0)
#@constraint(model, x5 <= 0)
@constraint(model, x6 <= 0)
@constraint(model, x7 <= 0)
#@constraint(model, x8 <= 0)

JuMP.optimize!(model)

println("The objective value is: \nz = ", JuMP.objective_value(model))

println("The solution is: ")
println("x1 = ", JuMP.value(x1))
println("x2 = ", JuMP.value(x2))
println("x3 = ", JuMP.value(x3))
println("x4 = ", JuMP.value(x4))
println("x5 = ", JuMP.value(x5))
println("x6 = ", JuMP.value(x6))
println("x7 = ", JuMP.value(x7))
println("x8 = ", JuMP.value(x8))

println("Dual variables: ")
println("yAB = ", dual(yAB))
println("yAD = ", dual(yAD))
println("yBC = ", dual(yBC))
println("yBD = ", dual(yBD))
println("yCD = ", dual(yCD))
println("yDE = ", dual(yDE))
println("yEA = ", dual(yEA))

```

```
println("y1_ = ", dual(y1))  
println("y2_ = ", dual(y2))  
println("y3_ = ", dual(y3))
```